

The background of the slide is a grayscale image of a circuit board. It features a complex network of lines representing traces and several circular pads. A solid black horizontal bar spans the middle of the image, providing a high-contrast background for the text.

Writing methods and classes

In Java, classes have both *state* and *behavior*.

State is the instance variables (**also called fields**)

Behavior are the **methods**

The process of designing the state and behavior of classes is sometimes called *modeling*.

We are modeling an abstraction of the real world in our custom classes.

Modeling things from life

If we create a class to model a dog, we probably won't model each of the billions of atoms inside the dog

<https://xkcd.com/505/>

Modeling things from life

What might be the *state* and *behavior* of a dog or a cat?

- Do the behaviors affect the state?
- Do two classes interact with each other?
(i.e. the class human can have a behavior that pets an instance of the class dog)

An amusing example from past years:

Terminator robot

- State:
 - name
 - number of weapons
 - type of sunglasses
- Behavior:
 - acquire sunglasses
 - eliminate Sarah Connor

Recall method signatures

- First line of a method, with name, arguments, and return type:

```
public void doSomething(int num, char c)
```

```
public String doOtherThing()
```

In general: `public returnType methodName(argument list)`

What is the signature of a String method `indexOf` that takes a char and returns the index where it occurs (or -1 if it doesn't occur)?

- A. `public void indexOf(char c, int index)`
- B. `public String indexOf(char c)`
- C. `public index indexOf(String s, char c)`
- D. `public int indexOf(char c)`
- E. None of the above

What is the signature of a String method `indexOf` that takes a char and returns the index where it occurs (or -1 if it doesn't occur)?

- A. `public void indexOf(char c, int index)`
- B. `public String indexOf(char c)`
- C. `public index indexOf(String s, char c)`
- D. `public int indexOf(char c)`
- E. None of the above

What is the signature of a method `dinnerTime` that takes a time such as “5:23pm” and returns whether it is an appropriate time for dinner?

- A. `public int dinnerTime(Time t)`
- B. `public String dinnerTime(Time t)`
- C. `public boolean dinnerTime(String s)`
- D. `public Time dinnerTime(double t)`
- E. None of the above

What is the signature of a method `dinnerTime` that takes a time such as “5:23pm” and returns whether it is an appropriate time for dinner?

- A. `public int dinnerTime(Time t)`
- B. `public String dinnerTime(Time t)`
- C. `public boolean dinnerTime(String s)`
- D. `public Time dinnerTime(double t)`
- E. None of the above

What is the signature of a Notebook method add that takes a String representing a note and adds it to the end of the book?

- A. `public Notebook add(String note)`
- B. `public Notebook add(Note note)`
- C. `public Note add(String note)`
- D. `public void add(Note note)`
- E. None of the above

What is the signature of a Notebook method add that takes a String representing a note and adds it to the end of the book?

- A. `public Notebook add(String note)` => this could also work
- B. `public Notebook add(Note note)`
- C. `public Note add(String note)`
- D. `public void add(Note note)`
- E. None of the above `public void add(String note)`

```
import java.util.*;

class Notebook {

    ArrayList<String> notes = new ArrayList<>();

    public void add(String note) {
        notes.add(note);
    }

    public Notebook add2(String note) {
        notes.add(note);
        return this; // this is means the current instance the method was called on
    }

    public static void main(String[] args) {
        Notebook book = new Notebook();

        book.add("Oh I have a great idea!");
        book.add("Let's cast magic spells");

        book.add2("Another idea!").add2("even more magic"); // chaining together method calls
    }
}
```

Encapsulation

What data do you think is encapsulated inside a String?

- A. It's a magical thing called a String that you should not question
- B. Array of ints that are converted to ASCII values
- C. Array of chars
- D. Collection of Strings of length 1
- E. None of the above

Encapsulation

What data do you think is encapsulated inside a String?

- A. It's a magical thing called a String that you should not question
- B. Array of ints that are converted to ASCII values
- C. Array of chars**
- D. Collection of Strings of length 1
- E. None of the above

But A isn't entirely wrong either....

A String encapsulates chars

The String is a class, a blueprint that can encapsulate some data. The data, or *state*, is the stuff being encapsulated. We can also define some behaviors, such as the `length()` method.

Encapsulation lets us package together lots of data and behaviors into something bigger.

An object is an instance of a class.

```
String str = new String();
```

String is the class

str is the object or instance of the class

Let's model! Pick something from class

State?

- what data must it track?

Behavior?

- what operations or functions does it have?
- do these operations affect the state?

Let's model: `class Vehicle`

State?

- what data must it track?

Behavior?

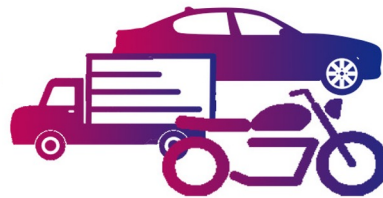
- what operations or functions does it have?
- do these operations affect the state?

class = blueprint for a type (e.g. Vehicle)

instance of class = variable of type (e.g. Vehicle car)

```
public class Vehicle{  
    private int numberOfWheels;  
    private String brandName, color;  
    private double price;  
    public void start();  
    public void changeGear();  
}
```

blue print of an object



Vehicle **car** = new Vehicle();



Vehicle **bike** = new Vehicle();



Vehicle **truck** = new Vehicle();



actual object

www.btechsmartclass.com

```
car.numberOfWheels = 4;  
car.start();  
car.changeGear();
```

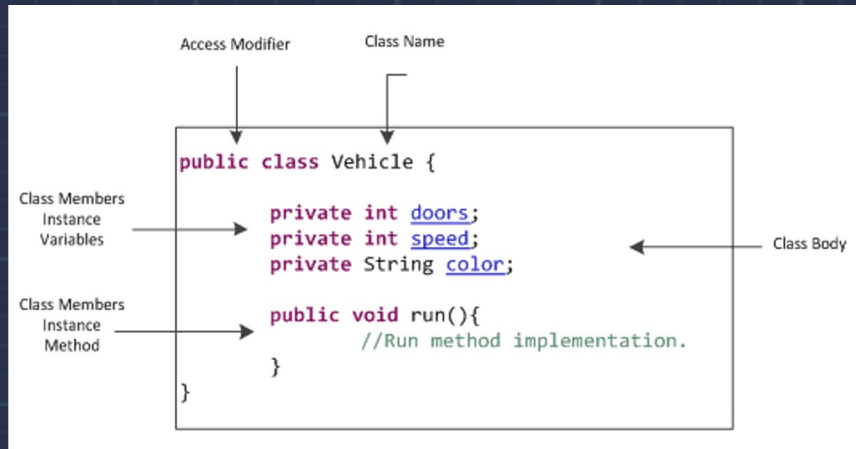
But we can do so much more!

(For example, we might find it really useful to specify what gear we are changing to...)

class Vehicle (uber simplified)

- State
 - speed
 - color
 - doors
- Behavior
 - run

class = blueprint for a type (e.g. Vehicle)
instance of class = variable of type (e.g. Vehicle car)



The class blueprint has:

- **a name** (Vehicle)
- **fields** (doors, speed, color)
 - **instance fields/variables** (describe instances)
 - **static fields** (describe class itself)
- **methods** (run)
 - **instance methods** (what instances can do)
 - **static methods** (what class itself can do)
- **constructors** (a factory to make instances using keyword "new")

```
public Vehicle() { // named like the class, without class word  
    //empty, so it sets all instance variables to default values  
}
```

```
public Vehicle(int d, int s, int c) { //non-empty constructor takes in info  
    doors = d;  
    speed = s;  
    color = c;  
}
```

Let's revisit your first Java program class

```
public class HelloWorld {  
    public static void main(String[] args) {  
  
        System.out.println("Hello");  
  
    }  
}
```

```
public class HelloWorld {  
    //---- fields (instance variables) ----  
    private String greeting;  
    //--- constructor (makes instances) ---  
    public HelloWorld(){  
        greeting = "Hello";  
    }  
    //----- methods -----  
    public void sayHello(){  
        System.out.println(greeting);  
    }  
    //----- our special method: main() -----  
    public static void main(String[] args) {  
        HelloWorld program = new HelloWorld();  
        program.sayHello();  
    }  
}
```

// any program with access to my class HelloWorld can
now make instances with "new" and use sayHello()
(no need to know how to print or what to say).
We are encapsulating functionality for later use!

We can have infinite instances of Vehicle

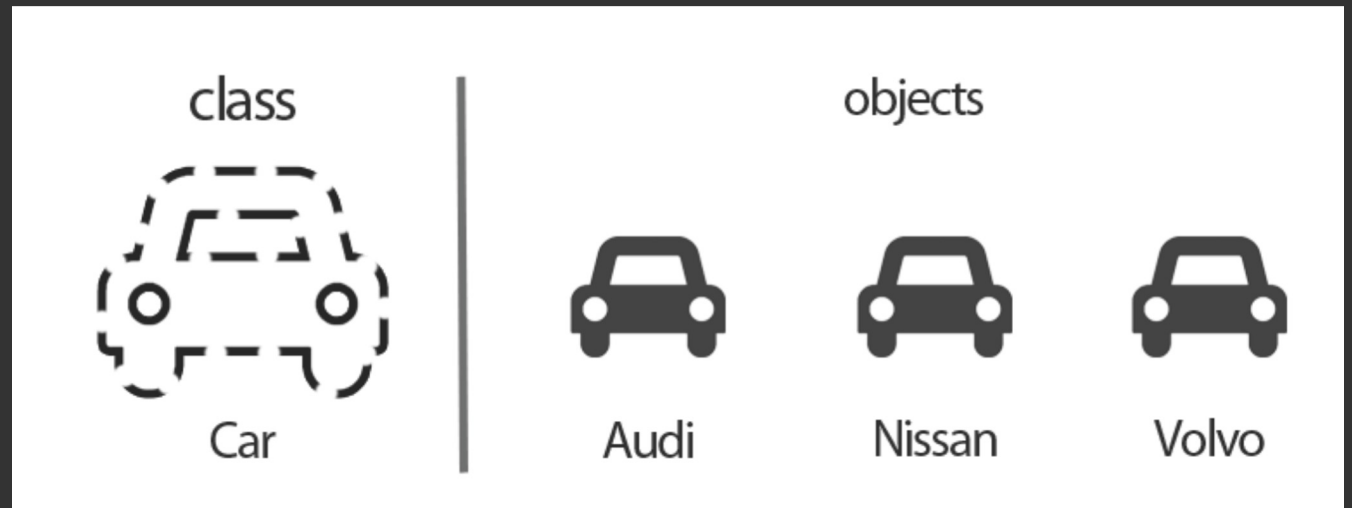
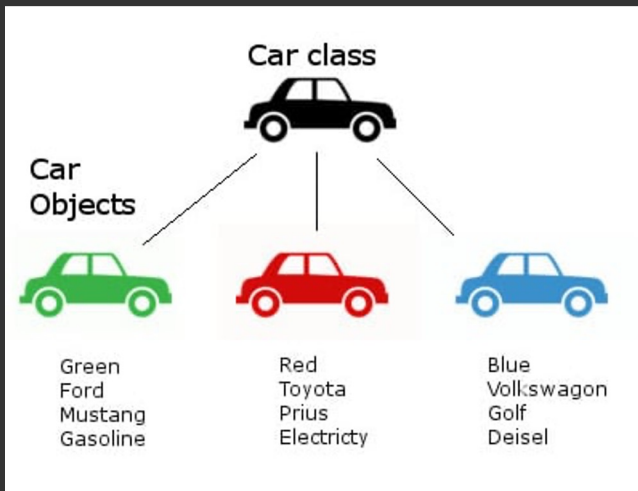
- The whole point of a class is to **encapsulate data and behavior** so that we have pre-made blueprints for instances of that class (things of that type)
- The **state** and **behavior** affect a particular **instance of Vehicle**
- The **instances of Vehicle** have different states (color, tank capacity, etc.)
- **Instances methods** must be called on instances, and will have different effects because the state of each instance was different.

if you call run() on a car that has gas, it'll run

if you call run() on an empty tank, then you might need to run fuelUp()

instead

We can have infinite instances of Vehicle
Two ways to think about this



class Vehicle (one possible implementation)

```
public class Vehicle{
    private double fuel; //current fuel, in gallons
    private double tankCapacity; //max gallons

    public Vehicle(double tankCapacity) { //creates new car filled to capacity
        this.tankCapacity = tankCapacity;
        this.fuel = this.tankCapacity;
    }

    public void fuelUp (double gallons) {
        fuel += gallons;
        if (fuel > tankCapacity) //maxed out?
            fuel = tankCapacity;
    }

    public void run() {
        this.fuel = this.fuel - 0.1; //expend 1/10 of a gallon each time we move
    }
}
```

class Vehicle (one possible implementation)

```
public class Vehicle{
    private double fuel; //current fuel, in gallons
    private double tankCapacity; //max gallons

    public Vehicle() { //creates new car filled to capacity
        this.fuel = this.tankCapacity;
    }

    public void fuelUp (double gallons) {
        fuel += gallons;
        if (fuel > tankCapacity) //maxed out?
            fuel = tankCapacity;
    }

    public void run() {
        this.fuel = this.fuel - 0.1; //expend 1/10 of a gallon each time we move
    }
}
```

Writing a class creates a *new reference type*!

Name of the class must be a legal Java *identifier* (plus follow the first letter capitalization convention)

- yes for classes
- no for variables/instances

example: Turtle turtle

↑ ↑
class instance

class Vehicle (one possible implementation)

```
public class Vehicle{
    private double fuel; //current fuel, in gallons
    private double tankCapacity; //max gallons

    public Vehicle() { //creates new car filled to capacity
        this.fuel = this.tankCapacity;
    }

    public void fuelUp (double gallons) {
        fuel += gallons;
        if (fuel > tankCapacity) //maxed out?
            fuel = tankCapacity;
    }

    public void run() {
        this.fuel = this.fuel - 0.1; //expend 1/10 of a gallon each time we move
    }
}
```

Instance Variables
(or *fields* or *data*)
are declared like any
other variables

They describe the state of
the class *instance*.

location: *inside* the class,
but *outside* any method

class Vehicle (one possible implementation)

```
public class Vehicle{
    private double fuel; //current fuel, in gallons
    private double tankCapacity; //max gallons

    public Vehicle() { //creates new car filled to capacity (to specified or default values)
        this.fuel = this.tankCapacity;
    }

    public void fuelUp (double gallons) {
        fuel += gallons;
        if (fuel > tankCapacity) //maxed out?
            fuel = tankCapacity;
    }

    public void run() {
        this.fuel = this.fuel - 0.1; //expend 1/10 of a gallon each time we move
    }
}
```

Constructor: special method that creates new instances

- no return value
- name matches name of class exactly
- sets initial values of instance variables (to specified or default values)

A class can have multiple constructors (with varying parameters so we can choose what to specify vs what to leave as default);

if no constructor is specified, Java provides a default empty constructor
`public Vehicle() { }`

class Vehicle (one possible implementation)

```
public class Vehicle{
    private double fuel; //current fuel, in gallons
    private double tankCapacity; //max gallons

    public Vehicle() { //creates new car filled to capacity
        this.fuel = this.tankCapacity;
    }

    public void fuelUp (double gallons) {
        fuel += gallons;
        if (fuel > tankCapacity) //maxed out?
            fuel = tankCapacity;
    }

    public void run() {
        this.fuel = this.fuel - 0.1; //expend 1/10 of a gallon each time we move
    }
}
```

instance methods: can access instance variables
(we use public methods to access private variables)

These do not use keyword static, so they are instance methods, not a static methods (a.k.a. functions)

class Vehicle (one possible implementation)

```
public class Vehicle{
    private double fuel; //current fuel, in gallons
    private double tankCapacity; //max gallons

    public Vehicle() { //creates new car filled to capacity
        this.fuel = this.tankCapacity;
    }

    public void fuelUp (double gallons) {
        fuel += gallons;
        if (fuel > tankCapacity) //maxed out?
            fuel = tankCapacity;
    }

    public void run() {
        this.fuel = this.fuel - 0.1; //expend 1/10 of a gallon each time we move
    }
}
```

instance methods: can access instance variables
(we use public methods to access private variables)

These do not use keyword static, so they are instance methods, not a static methods (a.k.a. functions)

static fields and methods are called on the Class itself, so they are only suitable for states and behavior that do not vary across class Instances
e.g. all vehicles are intended for transport, so:
static String purpose = "transport";

Notice how we are referencing the class fields

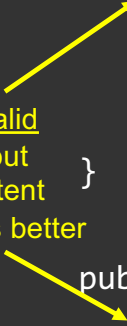
```
public class Vehicle{
    private double fuel;
    private double tankCapacity; //max gallons

    public Vehicle() {
        this.fuel = this.tankCapacity; //fill up
    }

    public void fuelUp (double gallons) {
        fuel += gallons;
        if (fuel > tankCapacity) //maxed out?
            fuel = tankCapacity;
    }

    public void run() {
        this.fuel = this.fuel - 0.1; //expend 1/10 of a gallon each time we move
    }
}
```

both valid
here, but
consistent
style is better



Notice how we are referencing the class fields

```
public class Vehicle{
    private double fuel;
    private double tankCapacity; //max gallons

    public Vehicle() {
        this.fuel = this.tankCapacity; //fill up
    }

    public void fuelUp (double gallons) {
        fuel += gallons;
        if (fuel > tankCapacity) //maxed out?
            fuel = tankCapacity;
    }

    public void run() {
        this.fuel = this.fuel - 0.1; //expend 1/10 of a gallon each time we move
    }
}
```

both valid
here, but
consistent
style is better

public methods of a class can access
private fields of the same class

syntax options: both valid, but with caveats...

- 1) `variableName` example: `fuel`
- 2) `this.variableName` example: `this.fuel`

true for both fields AND methods

`this` refers to this instance of the class

when calling from another class:

```
turtle.forward(200);
```

when calling from inside its own class (Turtle):

```
forward(200);
```

```
this.forward(200);
```


Accessing fields: `this.variableName` vs `variableName`



```
public void fuelUp (double gallons) {  
    fuel = fuel + gallons;  
    if (this.fuel > tankCapacity) //maxed out?  
        this.fuel = tankCapacity;  
}
```

public methods of a class can access
private fields of the same class

syntax:

`variableName`

example: `fuel`

`this.variableName` example:

`this.fuel`

both valid, just be consistent

I prefer `this.variableName`
because it makes naming parameters easier
and more straightforward



```
public void fuelUp (double fuel) {  
    this.fuel = this.fuel + fuel;  
    if (this.fuel > tankCapacity) //maxed out?  
        this.fuel = tankCapacity;  
}
```



```
public void fuelUp (double fuel) {  
    fuel = fuel + fuel; //WILL NOT WORK CORRECTLY!!!  
    if (this.fuel > tankCapacity) //maxed out?  
        this.fuel = tankCapacity;  
}
```

What is the signature of a constructor for the class Cat that takes a String which is the cat's name?

- A. `public Cat(String name)`
- B. `public Cat Cat(String name)`
- C. `public Cat(Name name)`
- D. `public Cat Cat(Name name)`
- E. None of the above

What is the signature of a constructor for the class Cat that takes a String which is the cat's name?

- A. public Cat(String name)
- B. public Cat Cat(String name)
- C. public Cat(Name name)
- D. public Cat Cat(Name name)
- E. None of the above

Simple class: WaterBottle

- State?
- Behavior?

Let's plan our desired behavior and how it relates to the state before writing any code

- **maxcap** of 20 oz
- start state: empty, meaning **current** amount is 0oz
- **add** some amount (5oz)
 - effect: increase current
 - edge case: overflow
 - max out amount
 - do we want a return value?
 - zero if no overflow
 - amount over capacity if overflow
- **remove** some amount (3oz)
 - effect: update current
 - edge case: insufficient current amount
 - zero out the amount
 - return value?
 - return the requested amount
 - if insufficient current amount, return everything left over

```
public class WaterBottle {
    private int capacity;
    private int current;

    public WaterBottle(int capacity) {
        this.capacity = capacity;
        this.current = 0;
    }

    public int add(int added) {
        this.current += added;
        if (this.current > this.capacity) {
            int excess = this.current - this.capacity;
            this.current = this.capacity;
            return excess;
        }
        return 0;
    }

    public int remove(int removed) {
        this.current -= removed;
        if (this.current < 0) {
            int actuallyRemoved = removed + this.current;
            this.current = 0;
            return actuallyRemoved;
        }
        return removed;
    }
}
```

```
public class WaterBottle {  
    private int capacity;  
    private int current;  
  
    public WaterBottle(int capacity) {  
        this.capacity = capacity;  
        this.current = 0;  
    }  
  
    public int add(int added) {  
        this.current += added;  
        if (this.current > this.capacity) {  
            int excess = this.current - this.capacity;  
            this.current = this.capacity;  
            return excess;  
        }  
        return 0;  
    }  
  
    public int remove(int removed) {  
        this.current -= removed;  
        if (this.current < 0) {  
            int actuallyRemoved = removed + this.current;  
            this.current = 0;  
            return actuallyRemoved;  
        }  
        return removed;  
    }  
}
```

What is the highlighted variable?

- A: formal parameter
- B: actual parameter
- C: local variable
- D: instance variable (or field)
- E: static variable

```
public class WaterBottle {  
    private int capacity;  
    private int current;  
  
    public WaterBottle(int capacity) {  
        this.capacity = capacity;  
        this.current = 0;  
    }  
  
    public int add(int added) {  
        this.current += added;  
        if (this.current > this.capacity) {  
            int excess = this.current - this.capacity;  
            this.current = this.capacity;  
            return excess;  
        }  
        return 0;  
    }  
  
    public int remove(int removed) {  
        this.current -= removed;  
        if (this.current < 0) {  
            int actuallyRemoved = removed + this.current;  
            this.current = 0;  
            return actuallyRemoved;  
        }  
        return removed;  
    }  
}
```

What is the highlighted variable?

- A: formal parameter
- B: actual parameter
- C: local variable
- D: instance variable (or field)**
- E: static variable

Why? It is inside a class but outside of any method.


```
public class WaterBottle {  
    private int capacity;  
    private int current;  
  
    public WaterBottle(int capacity) {  
        this.capacity = capacity;  
        this.current = 0;  
    }  
  
    public int add(int added) {  
        this.current += added;  
        if (this.current > this.capacity) {  
            int excess = this.current - this.capacity;  
            this.current = this.capacity;  
            return excess;  
        }  
        return 0;  
    }  
  
    public int remove(int removed) {  
        this.current -= removed;  
        if (this.current < 0) {  
            int actuallyRemoved = removed + this.current;  
            this.current = 0;  
            return actuallyRemoved;  
        }  
        return removed;  
    }  
}
```

What is the highlighted variable?

- A: formal parameter
- B: actual parameter
- C: local
- D: instance
- E: static

```
public class WaterBottle {  
    private int capacity;  
    private int current;  
  
    public WaterBottle(int capacity) {  
        this.capacity = capacity;  
        this.current = 0;  
    }  
  
    public int add(int added) {  
        this.current += added;  
        if (this.current > this.capacity) {  
            int excess = this.current - this.capacity;  
            this.current = this.capacity;  
            return excess;  
        }  
        return 0;  
    }  
  
    public int remove(int removed) {  
        this.current -= removed;  
        if (this.current < 0) {  
            int actuallyRemoved = removed + this.current;  
            this.current = 0;  
            return actuallyRemoved;  
        }  
        return removed;  
    }  
}
```

What is the highlighted variable?

A: formal parameter

B: actual parameter

C: local

D: instance

E: static

Why? It's in parens in a method header

```
public class WaterBottle {  
    private int capacity;  
    private int current;  
  
    public WaterBottle(int capacity) {  
        this.capacity = capacity;  
        this.current = 0;  
    }  
  
    public int add(int added) {  
        this.current += added;  
        if (this.current > this.capacity) {  
            int excess = this.current - this.capacity;  
            this.current = this.capacity;  
            return excess;  
        }  
        return 0;  
    }  
  
    public int remove(int removed) {  
        this.current -= removed;  
        if (this.current < 0) {  
            int actuallyRemoved = removed + this.current;  
            this.current = 0;  
            return actuallyRemoved;  
        }  
        return removed;  
    }  
}
```

What type of method is **WaterBottle()**?

- A: instance method
- B: static method
- C: constructor
- D: local method
- E: reference method

```

public class WaterBottle {
    private int capacity;
    private int current;

    public WaterBottle(int capacity) {
        this.capacity = capacity;
        this.current = 0;
    }

    public int add(int added) {
        this.current += added;
        if (this.current > this.capacity) {
            int excess = this.current - this.capacity;
            this.current = this.capacity;
            return excess;
        }
        return 0;
    }

    public int remove(int removed) {
        this.current -= removed;
        if (this.current < 0) {
            int actuallyRemoved = removed + this.current;
            this.current = 0;
            return actuallyRemoved;
        }
        return removed;
    }
}

```

What type of method is **WaterBottle()**?

- A: instance method
- B: static method
- C: constructor
- D: local method (not a thing)
- E: reference method (not a thing)

Why? No return type and same name as the class

```
public class WaterBottle {
    private int capacity;
    private int current;

    public WaterBottle(int capacity) {
        this.capacity = capacity;
        this.current = 0;
    }

    public int add(int added) {
        this.current += added;
        if (this.current > this.capacity) {
            int excess = this.current - this.capacity;
            this.current = this.capacity;
            return excess;
        }
        return 0;
    }

    public int remove(int removed) {
        this.current -= removed;
        if (this.current < 0) {
            int actuallyRemoved = removed + this.current;
            this.current = 0;
            return actuallyRemoved;
        }
        return removed;
    }
}
```

What type of method is **add()**?

- A: instance method
- B: static method
- C: constructor
- D: local method
- E: reference method

```

public class WaterBottle {
    private int capacity;
    private int current;

    public WaterBottle(int capacity) {
        this.capacity = capacity;
        this.current = 0;
    }

    public int add(int added) {
        this.current += added;
        if (this.current > this.capacity) {
            int excess = this.current - this.capacity;
            this.current = this.capacity;
            return excess;
        }
        return 0;
    }

    public int remove(int removed) {
        this.current -= removed;
        if (this.current < 0) {
            int actuallyRemoved = removed + this.current;
            this.current = 0;
            return actuallyRemoved;
        }
        return removed;
    }
}

```

What type of method is **add()**?

- A: instance method
- B: static method
- C: constructor
- D: local method (still not a thing)
- E: reference method (still not a thing)

Why? looks like a method (has "public", return type, a name, and parameters) but does not have "static" in the header

```

public class WaterBottle {
    private int capacity;
    private int current;

    public WaterBottle(int capacity) {
        this.capacity = capacity;
        this.current = 0;
    }

    public int add(int added) {
        this.current += added;
        if (this.current > this.capacity) {
            int excess = this.current - this.capacity;
            this.current = this.capacity;
            return excess;
        }
        return 0;
    }

    public int remove(int removed) {
        this.current -= removed;
        if (this.current < 0) {
            int actuallyRemoved = removed + this.current;
            this.current = 0;
            return actuallyRemoved;
        }
        return removed;
    }
}

```

What type of method is **remove()**?

- A: instance method
- B: static method
- C: constructor
- ~~D: local method~~
- ~~E: reference method~~

```

public class WaterBottle {
    private int capacity;
    private int current;

    public WaterBottle(int capacity) {
        this.capacity = capacity;
        this.current = 0;
    }

    public int add(int added) {
        this.current += added;
        if (this.current > this.capacity) {
            int excess = this.current - this.capacity;
            this.current = this.capacity;
            return excess;
        }
        return 0;
    }

    public int remove(int removed) {
        this.current -= removed;
        if (this.current < 0) {
            int actuallyRemoved = removed + this.current;
            this.current = 0;
            return actuallyRemoved;
        }
        return removed;
    }
}

```

What type of method is **remove()**?

- A: instance method
- B: static method
- C: constructor
- D: ~~local method~~
- E: ~~reference method~~

Why? Same reasons as for add():
looks like a method (has "public",
return type, a name, and
parameters) but does not have
"static" in the header


```

public class WaterBottle {
    //----- fields -----
    private int capacity;
    private int current;
    //----- constructors -----
    public WaterBottle(int capacity) {
        this.capacity = capacity;
        this.current = 0;
    }
    //----- methods -----
    public int add(int added) {
        this.current += added;
        if (this.current > this.capacity) {
            int excess = this.current - this.capacity;
            this.current = this.capacity;
            return excess;
        }
        return 0;
    }

    public int remove(int removed) {
        this.current -= removed;
        if (this.current < 0) {
            int actuallyRemoved = removed + this.current;
            this.current = 0;
            return actuallyRemoved;
        }
        return removed;
    }
}

```

You can always add some sectioning comment lines to separate these definitions into groups

You can also separate chunks of code in this way to group major problem-solving steps!

```
public class WaterBottle {
    private int capacity;
    private int current;

    public WaterBottle(int capacity) {
        this.capacity = capacity;
        this.current = 0;
    }

    public int add(int added) {
        this.current += added;
        if (this.current > this.capacity) {
            int excess = this.current - this.capacity;
            this.current = this.capacity;
            return excess;
        }
        return 0;
    }

    public int remove(int removed) {
        this.current -= removed;
        if (this.current < 0) {
            int actuallyRemoved = removed + this.current;
            this.current = 0;
            return actuallyRemoved;
        }
        return removed;
    }
}
```

What is the highlighted variable?

- A: formal parameter
- B: actual parameter
- C: local variable
- D: instance variable
- E: static variable

```

public class WaterBottle {
    private int capacity;
    private int current;

    public WaterBottle(int capacity) {
        this.capacity = capacity;
        this.current = 0;
    }

    public int add(int added) {
        this.current += added;
        if (this.current > this.capacity) {
            int excess = this.current - this.capacity;
            this.current = this.capacity;
            return excess;
        }
        return 0;
    }

    public int remove(int removed) {
        this.current -= removed;
        if (this.current < 0) {
            int actuallyRemoved = removed + this.current;
            this.current = 0;
            return actuallyRemoved;
        }
        return removed;
    }
}

```

What is the highlighted variable?

- A: formal parameter
- B: actual parameter
- C: local variable
- D: instance variable
- E: static variable

Why?

- created inside a block (body of if statement)
- only lives during that block

Write a class Food with...

- A String attribute called type and an integer attribute called calories
- A constructor that takes values for these attributes and sets them
- A method toString that returns a String of the form “cheese (100 calories)”; the name followed by a parenthetical with the number of calories and the word “calories”
- A method getCalories that returns the number of calories

```
public class Food {  
    private String type; //type of food  
    private int calories; //its number of calories  
  
    public Food(String type, int calories) {  
        //constructor that takes values for the attributes  
        this.type = type;  
        this.calories = calories;  
    }  
  
    public String toString() {  
        return type + " (" + calories + " calories)";  
    }  
  
    public int getCalories() {  
        return calories;  
    }  
}
```

```
public class Fridge {  
    public static void main(String[] args) {  
        Food[] foods = new Food[5];  
        foods[0] = new Food("cheese", 100);  
        foods[1] = new Food("apple", 50);  
        foods[2] = new Food("banana", 200);  
        foods[3] = new Food("cookie", 500);  
        foods[4] = new Food("cake", 1000);  
  
        //print summary of available foods  
        System.out.println("You currently have the following:");  
        for(int x = 0; x < foods.length; x++)  
            System.out.println((x+1) + ". " + foods[x]);  
  
        //ask what they want  
        System.out.println("What would you like to eat?");  
        Scanner input = new Scanner(System.in);  
        int choice = input.nextInt();  
        System.out.println("Yum! You got " + foods[choice-1].getCalories() + " calories");  
    }  
}
```

```
public class Vehicle {
    private double fuel; //current fuel, in gallons
    private double tankCapacity; //max gallons

    public Vehicle(double tankCapacity) { //creates new car filled to capacity
        this.tankCapacity = tankCapacity;
        this.fuel = this.tankCapacity;
    }

    public Vehicle() {
        this.tankCapacity = 10;
        this.fuel = 0;
    }

    public void fuelUp (double gallons) {
        fuel += gallons;
        if (fuel > tankCapacity) //maxed out?
            fuel = tankCapacity;
    }

    public void run() {
        this.fuel = this.fuel - 0.1; //expend 1/10 of a gallon each time we move
    }

    public static void main(String[] args)
    {
        double tankCapacity = 37;
        Vehicle v = new Vehicle(tankCapacity);

        Vehicle v2 = new Vehicle(12);
    }
}
```